Noname manuscript No.
(will be inserted by the editor)

Multi-task Real-Time 3D LiDAR Perception with Attention-Enhanced MobilePIXOR for Obstacle Segmentation and Pedestrian Detection in Autonomous Robots

Hoang N. Tran · Cong T. Nguyen · Truong T. Duong · Minh L. Nguyen · Vinh D. Nguyen ·

Received: date / Accepted: date

Abstract This paper presents a real-time 3D LiDAR perception framework, enhanced with an Attention-Driven MobilePIXOR model for obstacle segmentation and pedestrian detection in autonomous robots. The proposed multi-task model leverages the MobilePIXOR architecture with depthwise separable convolutions and integrates Convolutional Block Attention Module (CBAM), enhancing object detection precision to 96.80% and improving IoU for obstacle segmentation to 56.50%. A heatmap header system, incorporating Gaussian heatmaps, is used to predict object center locations, further improving detection accuracy. The framework converts LiDAR data into structured voxel grids, optimizing spatial awareness and enabling the differentiation between drivable and non-drivable areas. An integrated loss function is introduced to jointly optimize both detection and segmentation tasks. Additionally, the novel *Lidar3D_SEG&DET* dataset was developed to evaluate the framework's effectiveness. Experimental results demonstrate that the enhanced MobilePIXOR model with CBAM outperforms other architectures in real-time performance, making it ideal for autonomous navigation in dynamic environments.

Keywords Robot navigation, Pedestrian detection, MobilePIXOR, Gaussian heatmap, Point cloud, Semantic segmentation.

Hoang N. Tran, Cong T. Nguyen, Truong T. Duong, Minh L. Nguyen, and Vinh D. Nguyen
FPT University, Can Tho, 94000, VietNam.
E-mail: hoang2531992@gmail.com; congnt.ce191449@gmail.com; TruongDTCE182403@fpt.edu.vn; leminhhu.edu@gmail.com; VinhND29@fpt.edu.vn

1 Introduction

Autonomous robots operating in dynamic environments rely heavily on accurate perception and navigation systems to ensure safe and efficient operation. LiDAR technology has facilitated the rise of 3D point clouds as a critical data source for interpreting and understanding complex environments. One of the most crucial tasks for autonomous systems is the simultaneous detection of obstacles and pedestrians, which is vital for both safe navigation and human-robot interaction. Addressing these challenges requires robust frameworks that can perform both obstacle segmentation and pedestrian detection in real-time.

Traditional approaches, such as VoxelNet [1] , [2] and PointPillars [3], typically treat obstacle detection and segmentation tasks independently. These methods utilize voxel grids or pillar-based representations to process 3D point cloud data. While these approaches deliver competitive performance in object detection, they often suffer from high computational complexity and struggle to achieve real-time segmentation, particularly in dense urban environments.

Recent advancements have focused on unifying detection and segmentation tasks within a single multi-task learning framework. For example, Zou et al. [4] introduced a LiDAR-based multi-task learning framework designed to handle both static obstacles and dynamic agents such as pedestrians, improving processing efficiency by using a shared feature extraction backbone. This shared backbone enhances computational efficiency, though challenges remain in environments with fine details, where issues such as motion blur and occlusion in pedestrian-heavy scenes still reduce segmentation accuracy [5, 6].

Segmentation-specific approaches, such as RangeNet++ [7] and SalsaNext [8], have demonstrated remarkable success in segmenting large-scale 3D point clouds for autonomous driving applications. However, these methods often lack the fine-granularity necessary for detecting small, dynamic objects, such as pedestrians, which are crucial for safe navigation in urban environments. Moreover, these models are generally optimized for outdoor environments and do not perform as well in complex pedestrian environments where finer details are required.

A major limitation of current multi-task frameworks is achieving real-time performance without sacrificing segmentation accuracy. Many state-of-the-art approaches, such as those using large backbone architectures like ResNet [9], require significant computational resources, rendering them less suitable for real-time applications. Additionally, multi-task learning frameworks frequently suffer from task interference, where the optimization of one task (e.g., object detection) degrades the performance of another task (e.g., pedestrian segmentation) [10].

To address these challenges, this paper proposes a novel multi-task framework that integrates obstacle segmentation and pedestrian detection into a single, unified network architecture designed for real-time 3D LiDAR perception. Our framework leverages advancements in lightweight neural networks, specifically based on the basic PIXOR [12] and MobilePIXOR [13], improving the model with depthwise separable convolutions combined with the CBAM attention mechanism to enhance both accuracy and computational efficiency.

The proposed framework, Multi-task Attention - Enhanced MobilePIXOR, utilizes a heatmap header system with Gaussian heatmaps to predict object center locations, optimizing the precision of pedestrian and obstacle detection. The framework processes LiDAR data by converting it into structured voxel grids, optimizing spatial awareness, and enabling efficient differentiation between drivable and non-drivable areas. A joint optimization loss function is introduced to balance detection and segmentation tasks, minimizing task interference while maintaining high performance.

We validate our approach by developing a custom dataset, *Lidar3D_SEG&DET*, specifically designed to evaluate the performance of multi-task frameworks in complex environments. The enhanced MobilePIXOR framework achieves real-time performance on autonomous robots navigating dynamic environments, meeting the stringent demands necessary for practical deployment.

The contributions of this paper are as follows:

- A novel multi-task framework, Multi-task Attention-Enhanced MobilePIXOR, is proposed to integrate real-time obstacle segmentation and pedestrian detection. The framework leverages the MobileNetV2 architecture with depthwise separable convolutions combined with the CBAM attention mechanism, allowing for efficient handling of both object detection and free-space segmentation tasks while significantly enhancing feature representation, reducing computational demands, and maintaining high performance.
- A heatmap header system, incorporating Gaussian heatmaps, is employed to predict object center locations, while the framework simultaneously converts LiDAR data into structured voxel grids. This approach optimizes spatial awareness, enhances the precision of pedestrian and obstacle detection, and facilitates the differentiation between drivable and non-drivable areas.
- An integrated loss function is introduced to optimize both object detection and segmentation tasks jointly, enhancing overall system performance.
- A custom dataset, *Lidar3D_SEG&DET*, is developed to comprehensively evaluate the proposed framework's performance in both real-world and synthetic environments, with experimental validation demonstrating significant improvements in accuracy and real-time performance when comparing the Multi-task Attention-Enhanced MobilePIXOR to state-of-the-art architectures for autonomous navigation.

The remainder of this paper is organized as follows: Section II reviews related work on object detection and segmentation, focusing on techniques used in LiDAR-based perception systems. Section III elaborates on the proposed Attention-Enhanced MobilePIXOR architecture, detailing the voxelization process, network design, and the integration of attention mechanisms, specifically the Convolutional Block Attention Module (CBAM), to enhance feature representation. Additionally, the section describes the use of Gaussian heatmaps for object detection and localization. Section IV presents experimental results on the custom dataset *Lidar3D_SEG&DET*, comparing MobilePIXOR's performance with state-of-the-art architectures and demonstrating the impact of attention mechanisms on detection and segmentation accuracy. Finally, Section V concludes the paper by summarizing the key contributions and discussing potential future directions, including enhancements for large-scale environments and sensor fusion strategies.

2 Multi-task LiDAR Perception and Related Work

In recent years, the field of LiDAR-based perception has made significant strides, particularly in the areas of object detection and free space segmentation. These tasks

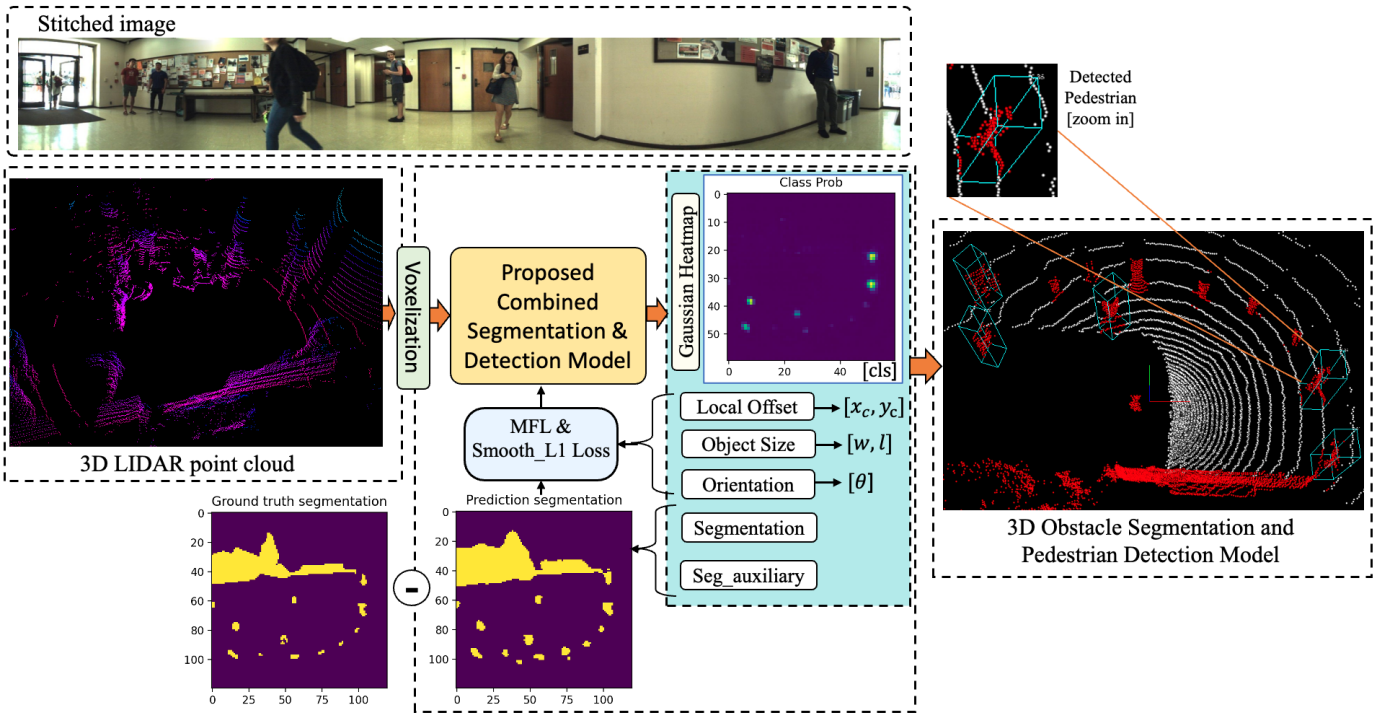


Fig. 1 Overview of the proposed combined 3D obstacle segmentation and pedestrian detection framework utilizing LiDAR point cloud data.

are critical in applications such as autonomous driving, where precise and real-time environmental understanding is essential. In this paper, we address the problem of 3D LiDAR-based object detection and segmentation, aiming to detect objects and segment free space within a given point cloud. The input to the system is a set of 3D points $P = \{p_1, p_2, \ldots, p_n\}$, where each point $p_i = (x_i, y_i, z_i)$ represents the spatial coordinates in 3D Euclidean space. Unlike other approaches that leverage additional attributes such as reflectance, our method focuses solely on spatial information, with a particular emphasis on mobile robot applications, where the height and z-value of obstacles are not considered critical.

The problem consists of two primary components: object detection and free space segmentation.

2.1 Object Detection

Given the input point cloud P, the objective of object detection is to identify and localize objects by predicting their bounding boxes and classifying them into predefined categories. For each detected object, the system outputs:

- A **class map** $C = \{c_1, c_2, \ldots, c_m\}$, where each $c_j \in \mathbb{R}^k$ is a probability distribution over k object classes.
- A **confidence score** $s_j \in [0, 1]$, representing the likelihood that the detected region corresponds to an object.
- A **bounding box** $b_j \in \mathbb{R}^5$, parameterized by its 3D position (x_j, y_j), dimensions (w_j, h_j), and orientation θ_j.

The object detection model can be expressed as a mapping:

$$f_{\text{det}} : P \to \{(c_j, s_j, b_j)\}_{j=1}^{m}, \tag{1}$$

where m is the number of detected objects, $c_j \in \mathbb{R}^k$, $s_j \in [0, 1]$, and $b_j \in \mathbb{R}^5$.

2.2 Free Space Segmentation

In parallel to object detection, the system performs free space segmentation, identifying regions in the point cloud corresponding to drivable or navigable areas. This segmentation is critical for tasks like autonomous driving, where distinguishing free space from obstacles is essential.

Free space segmentation is represented by a binary function:

$$g_{\text{seg}} : P \to \{0, 1\}^n, \tag{2}$$

where $g_{\text{seg}}(p_i) = 1$ indicates that the point p_i belongs to free space, and $g_{\text{seg}}(p_i) = 0$ indicates otherwise.

2.3 Joint Objective

The joint objective of the system is twofold:

1. To detect objects by predicting their class, confidence score, and bounding boxes.
2. To segment the input point cloud into free space and non-free space regions.

This combined task can be formulated as a unified mapping:

$$h : P \rightarrow \{(C, S, B), G\}, \tag{3}$$

where $C = \{c_j\}_{j=1}^{m}$, $S = \{s_j\}_{j=1}^{m}$, $B = \{b_j\}_{j=1}^{m}$, and $G = \{g_{\text{seg}}(p_i)\}_{i=1}^{n}$.

The model is designed to effectively handle both object detection and free space segmentation, ensuring high accuracy and real-time performance, which is crucial for applications like autonomous mobile robots.

2.4 Existing Approaches

The recent surge in multi-task learning (MTL) approaches for LiDAR-based perception has led to significant improvements in handling multiple tasks simultaneously. Voxel-based methods, such as VoxelNet [3], employ a 3D convolutional approach to process voxelized point clouds, making it one of the first frameworks to perform end-to-end learning for LiDAR-based object detection. However, due to the high computational cost, VoxelNet is often impractical for real-time applications.

To address these limitations, PointPillars [3] was introduced, which encodes point clouds into pseudo-images, allowing the use of 2D convolutional networks to detect objects efficiently. This approach is notable for its ability to strike a balance between speed and accuracy, outperforming other LiDAR-based methods in both 3D object detection and bird's eye view (BEV) object detection.

In addition to voxel-based methods, multi-task learning frameworks such as LiSD [14] propose integrating segmentation and detection into a single pipeline. LiSD uses a hierarchical feature collaboration module to share information between tasks, optimizing overall performance in segmentation and object detection. This approach significantly reduces the computational overhead by processing both tasks in parallel, making it suitable for real-time applications.

Chakraborty et al. [15] introduced an adaptive neighborhood selection technique for segmentation of LiDAR point clouds in urban environments. This method addresses the limitations of fixed neighborhood selection in traditional methods, improving the accuracy of point cloud segmentation, especially in complex urban areas with varying densities and geometries.

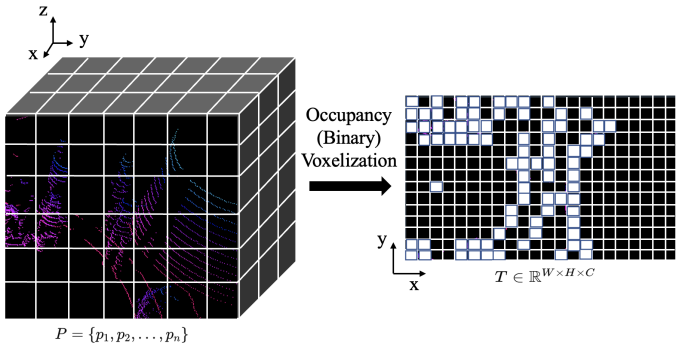


Fig. 2 Voxelization.

Khattar et al. [16] explored the use of cross-domain multi-task learning, where object detection and saliency estimation are learned jointly across different datasets. Their framework highlights the potential of multi-task learning to improve efficiency and memory utilization, making it an attractive solution for LiDAR-based perception tasks that require multiple objectives to be met simultaneously.

Despite these advancements, the aforementioned methods still face limitations in terms of accuracy and the ability to simultaneously handle both detection and segmentation tasks efficiently. Many voxel-based and multi-task learning approaches struggle with maintaining real-time performance while ensuring precise object localization and free-space segmentation, particularly in dynamic environments with dense point clouds. To overcome these challenges, we propose an Attention-Enhanced MobilePIXOR framework, which integrates lightweight network architectures, attention mechanisms, and an optimized multi-task loss function. By leveraging Convolutional Block Attention Module (CBAM) and Gaussian heatmaps, our approach significantly improves both detection precision and segmentation accuracy, while maintaining computational efficiency for real-time deployment in autonomous systems.

3 Methodology

4 Methodology

4.1 Voxelization for 3D LiDAR Object Detection

Voxelization is an essential preprocessing step for 3D LiDAR object detection, enabling efficient representation and computation of point cloud data. Given a set of 3D points $P = \{p_1, p_2, \ldots, p_n\}$, where each point $p_i = (x_i, y_i, z_i)$ is a LiDAR measurement with coordi-

nates in the 3D space, the primary objective of voxelization is to discretize the continuous 3D space into fixed-size volumetric units called voxels. Each voxel is represented by a binary value, indicating whether any points occupy it. This structured output facilitates subsequent processing in object detection pipelines by transforming the irregular point cloud data into a regular 3D grid.

4.1.1 Defining the Voxel Grid

The 3D space is partitioned into a grid of equally sized voxels, with the size of the grid determined by the spatial resolution of the voxel, defined by parameters v_x, v_y, v_z. Each voxel has dimensions $v_x \times v_y \times v_z$, and the range of the grid is bounded by the minimum and maximum coordinates in the LiDAR point cloud along all three axes $[x_{\min}, x_{\max}]$, $[y_{\min}, y_{\max}]$, and $[z_{\min}, z_{\max}]$.

This defines the total number of voxels in each dimension as:

$$N_x = \frac{x_{\max} - x_{\min}}{v_x}, N_y = \frac{y_{\max} - y_{\min}}{v_y}, N_z = \frac{z_{\max} - z_{\min}}{v_z} \tag{4}$$

where N_x, N_y, N_z represent the number of voxels along the x, y, and z axes, respectively.

4.1.2 Mapping Points to Voxels

Each point $p_i = (x_i, y_i, z_i)$ in the point cloud is assigned to a voxel in the grid. The voxel coordinates for a point are calculated as:

$$v_{x_i} = \left\lfloor \frac{x_i - x_{\min}}{v_x} \right\rfloor, v_{y_i} = \left\lfloor \frac{y_i - y_{\min}}{v_y} \right\rfloor, v_{z_i} = \left\lfloor \frac{z_i - z_{\min}}{v_z} \right\rfloor \tag{5}$$

This procedure maps the continuous 3D coordinates of each point to discrete voxel indices, facilitating efficient processing in object detection tasks.

4.1.3 Binary Voxel Grid Output with Occupancy Check

After mapping the points to their respective voxels, an occupancy check is performed. For each voxel in the grid, a binary value is assigned:

$$O(v_x, v_y, v_z) = \begin{cases} 1 & \text{if any point exists in the voxel} \\ 0 & \text{if no points are present} \end{cases} \tag{6}$$

This process results in a 3D occupancy grid, where each voxel is marked as either occupied (1) or empty (0). The output of the voxelization process is a 3D binary grid with dimensions $N_x \times N_y \times N_z$, where each voxel $O(v_x, v_y, v_z)$ contains either 0 or 1. This binary representation of the voxel grid provides a compact and structured form of LiDAR data, retaining essential spatial information while discarding unoccupied, computationally redundant regions. The binary voxel grid serves as the input for subsequent stages of the 3D object detection pipeline.

4.2 Input and Output of the Enhanced MobilePIXOR Network

The voxelization process transforms raw LiDAR point cloud data into a structured tensor representation, which is used as input for the MobilePIXOR network. The voxelization discretizes the 3D space into a fixed-size grid of voxels, resulting in a tensor $T \in \mathbb{R}^{W \times H \times C}$, where W and H represent the width and height of the grid, and C is the number of feature channels, encoding occupancy information. For this work, we use a single-channel tensor where $C = 1$, and each voxel is assigned a binary value representing whether it is occupied (1) or empty (0).

The input tensor T is processed by the MobilePIXOR network, a convolutional neural network (CNN) designed for efficient object detection and free space segmentation in 3D LiDAR scenes. The tensor T is structured as follows:

- W and H represent the horizontal resolution, corresponding to the bird's-eye view projection of the LiDAR point cloud.
- C is the number of channels, with $C = 1$, where each voxel contains a binary value representing occupancy:

$$T(v_x, v_y, 1) = \begin{cases} 1 & \text{if the voxel is occupied} \\ 0 & \text{if the voxel is empty} \end{cases} \tag{7}$$

The network operates as a function f, mapping the input tensor T to an output y ($y = f(T)$), where y consists of two components:

- **Bounding box detection**: The network predicts bounding boxes for detected objects in the scene, including spatial coordinates, dimensions, and orientation.
- **Free space segmentation**: The network generates a segmentation map identifying free space areas, crucial for navigation and path planning.

The MobilePIXOR network leverages lightweight operations, ensuring efficient processing that is suitable for real-time applications such as autonomous driving.

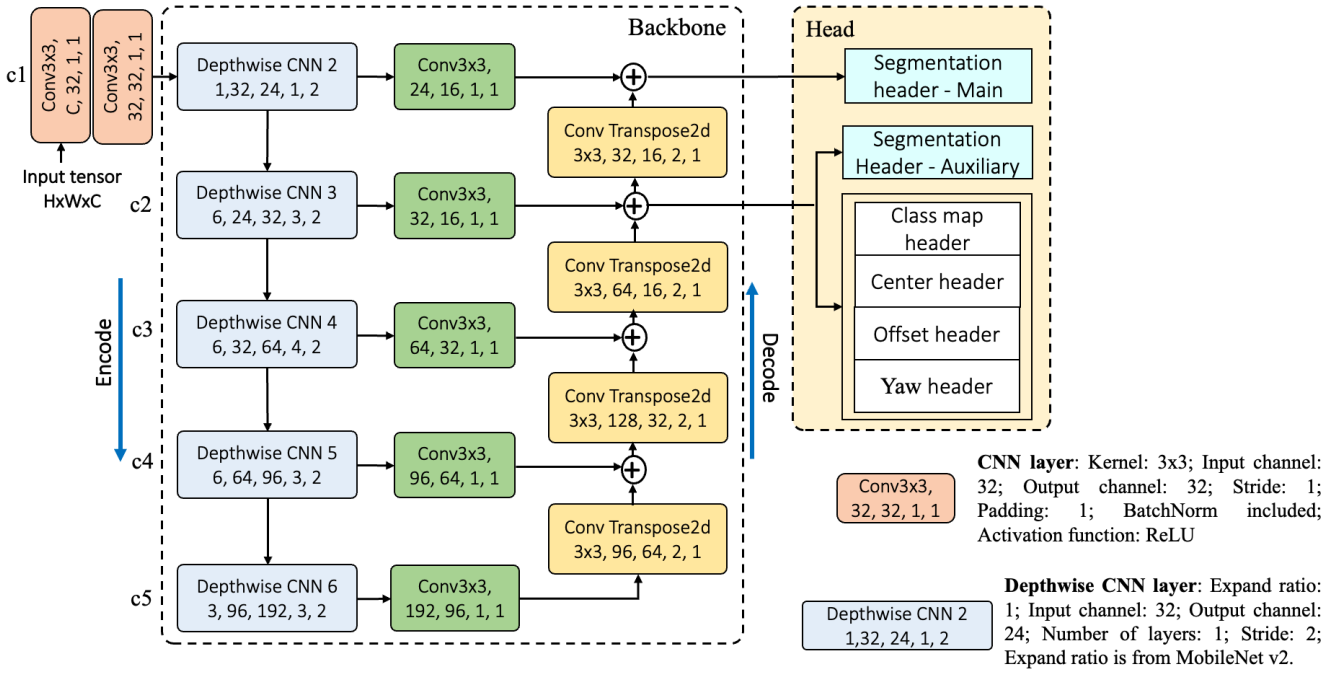


Fig. 3 Proposed Architecture: Multi-task Segmentation for Obstacle and Pedestrian Detection.

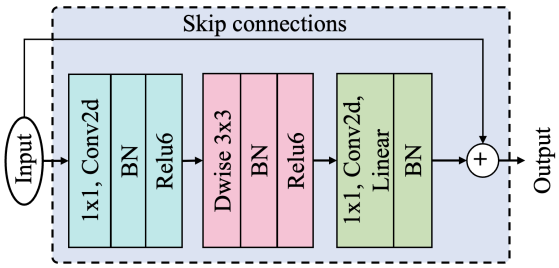


Fig. 4 Depthwise CNN Layer Architecture.

4.3 Multi-task Enhanced MobilePIXOR Architecture

4.3.1 Proposed Backbone

The MobilePIXOR network is designed for efficient 3D LiDAR object detection and free-space segmentation. The input to the network is a 3D tensor generated from the voxelization process, with dimensions $H \times W \times C$, where H and W represent the height and width of the voxel grid, and C is the number of channels encoding occupancy information. The backbone of the MobilePIXOR network, as illustrated in Fig. 3, is constructed using a series of convolutional and depthwise separable convolution layers to process the input tensor while maintaining computational efficiency effectively.

The initial layers of the network consist of two standard convolutional layers (CNN) that extract low-level features from the input tensor. Each of these layers applies a 3×3 kernel with a stride of 1 and padding of 1, followed by batch normalization and ReLU activation. These layers help preserve important spatial information and begin the downsampling process by reducing the resolution slightly.

Following the initial convolutional layers, the MobilePIXOR network utilizes six depthwise separable convolution layers, progressively increasing input and output channels to capture more complex features. Starting with 32 input channels, the network reduces the spatial dimensions and outputs 24 channels in the first layer. The second layer processes the data further, increasing the output to 32 channels. The third layer maintains the output at 32 channels. As the network continues to process the input, the fourth layer increases the number of channels to 64, followed by the fifth layer, which further increases it to 96. The final layer in this sequence expands the number of output channels to 192. Each layer uses a stride of 2 for downsampling, allowing the network to efficiently capture both local and global spatial features while maintaining computational efficiency, as illustrated in Fig. 3. These layers progressively downsample the feature maps, allowing the network to capture both local and global spatial information crucial for object detection and segmentation tasks. Skip connections, as shown in Fig. 4, are employed in each depthwise separable convolution layer to preserve high-resolution features that are crucial for the final upsampling path.

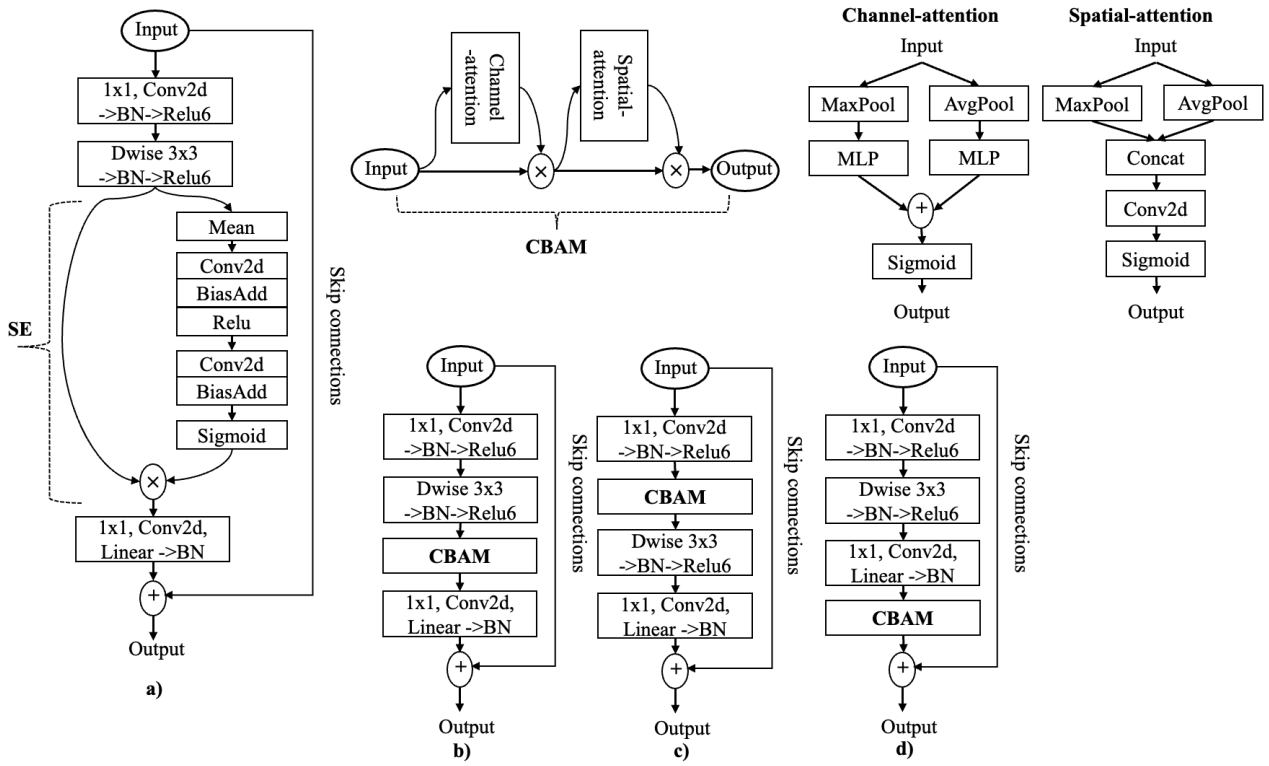


Fig. 5 Depthwise CNN Layer Architecture with SE and CBAM Attention Integration: (a) SE Attention after Depthwise Convolution (SE-MobilePIXOR), (b) CBAM after Depthwise Convolution (CBAM-MobilePIXORv1), (c) CBAM between Expand and Depthwise Convolution (CBAM-MobilePIXORv2), and (d) CBAM at the Final Position after BatchNorm (CBAM-MobilePIXORv3).

4.3.2 Attention Mechanisms

Additionally, Fig. 5 illustrates various methods for integrating attention mechanisms into the depthwise CNN layers: subfigure (a) depicts the SE attention structure, while subfigures (b), (c), and (d) present the CBAM attention mechanism integrated at different positions within the depthwise convolution layers: (b) after the depthwise convolution block, (c) between the expand channels and depthwise convolution layers, and (d) at the final position after the BatchNorm layer. These attention integration strategies aim to enhance the network's capability to focus on important spatial and channel-wise features. The impact of these strategies will be evaluated through experiments.

The final depthwise separable convolution layer is followed by a series of transposed convolution layers, also referred to as deconvolutions, that upsample the feature maps back to a higher resolution, as illustrated in Fig. 3. The transposed convolution layers use a 3×3 kernel with a stride of 2, progressively increasing the spatial resolution of the feature maps.

Lateral connections are incorporated to merge features from earlier layers with the upsampled features from deeper layers, ensuring that the network retains important high-resolution spatial details. These lateral connections use standard convolutional layers with a 3×3 kernel, stride 1, and padding 1, helping the network preserve crucial spatial information necessary for accurate object detection and free-space segmentation.

4.4 Proposed Header for Multi-task with Gaussian Heatmap and Auxiliary Segmentation

The MobilePIXOR network employs multiple output headers, each designed for a specific task in object detection and free space segmentation. These headers include the heatmap header for predicting object centers, class map, center, offset, and yaw headers for object detection, as well as segmentation headers for identifying drivable areas. The architecture is designed to handle both object detection and segmentation in a unified framework, providing robust multi-task learning capabilities.

The heatmap header predicts the location of object centers by outputting a heatmap of size $\frac{H}{4} \times \frac{W}{4} \times C$, where H and W represent the height and width of the input scene, and C denotes the number of object classes. At each spatial location (x, y), the heatmap value corresponds to the probability $p_i(x, y)$ that the

center of an object belonging to class i is located at that position. This probability is modeled using a Gaussian heatmap, which is centered at the true object location and defined as follows:

$$H_i(x,y) = \exp\left(-\frac{(x-x_c)^2+(y-y_c)^2}{2\sigma^2}\right) \tag{8}$$

where (x_c, y_c) represents the center of the object in class i, and σ is a parameter that controls the spread of the Gaussian heatmap, typically determined by the object size.

The Gaussian heatmap enables a probabilistic representation of object centers, where nearby locations are assigned non-zero values based on their distance from the true center. This helps mitigate the imbalance caused by variations in object sizes, as smaller objects tend to have fewer pixels representing their centers compared to larger objects. The use of Gaussian heatmaps "softens" this representation, allowing pixels near the true center to contribute to the overall detection process.

In addition to the heatmap header, the network features segmentation headers, which generate a segmentation map for identifying free space areas. The main segmentation header outputs the primary segmentation map, distinguishing between drivable and non-drivable areas, while the auxiliary segmentation header serves as a regularization branch to enhance segmentation performance. This auxiliary branch contributes to improved generalization by acting as a supplementary loss function during training, thereby refining the segmentation output and improving the network's ability to handle complex environments. By leveraging these headers, the MobilePIXOR network efficiently addresses multiple tasks simultaneously, combining object detection and segmentation in a manner that is both computationally efficient and highly accurate for real-time autonomous driving applications.

4.5 Loss Function for Handling Object Size Imbalance

As in the original architecture, the network predicts heatmaps for object centers, offset maps for center localization, and regression maps for bounding box size and yaw angles. The total loss function remains:

$$\mathcal{L}_{\text{Object-total}} = \mathcal{L}_{\text{heatmap}} + \mathcal{L}_{\text{regression}} \tag{9}$$

where $\mathcal{L}_{\text{heatmap}}$ and $\mathcal{L}_{\text{regression}}$ represent the heatmap loss and regression loss, respectively. These components ensure that the enhanced network maintains accuracy in both object detection and segmentation tasks while being computationally efficient enough for real-time edge deployment.

A common issue in object detection is the imbalance between small and large objects. Larger objects cover more spatial locations, leading to a bias in the loss function, which may focus more on larger objects. To alleviate this imbalance, the Gaussian heatmap ensures that smaller objects still have a meaningful representation by spreading the center location across multiple neighboring pixels.

Let $\mathcal{L}_{\text{heatmap}}$ represent the loss function for the heatmap prediction. The loss is computed as the pixel-wise focal loss between the predicted heatmap $\hat{H}_i(x,y)$ and the ground truth Gaussian heatmap $H_i(x,y)$ for each class i:

$$\mathcal{L}_{\text{heatmap}} = -\frac{1}{N}\sum_{i=1}^{C}\sum_{x=1}^{W/4}\sum_{y=1}^{H/4}\left(1-\hat{H}_i(x,y)\right)^{\alpha}\log\left(\hat{H}_i(x,y)\right)H_i(x,y) \tag{10}$$

where N is the number of object centers, α is a hyperparameter that controls the focusing of the focal loss, $\hat{H}_i(x,y)$ represents the predicted heatmap value for class i at location (x,y), and $H_i(x,y)$ denotes the ground truth Gaussian heatmap for class i at location (x,y).

This loss function encourages the network to focus on object center predictions while down-weighting the contribution of easy negatives (i.e., locations far from any object centers).

The heatmap header is a key component of the network, enabling accurate localization of object centers for each class while addressing the imbalance caused by varying object sizes.

1. Center Offset Regression

The network predicts an offset for the object center from the heatmap's coarse center prediction. Let the predicted offsets for the center at position (x,y) be denoted as $\Delta\hat{x}(x,y)$ and $\Delta\hat{y}(x,y)$. The true offsets, $\Delta x(x,y)$ and $\Delta y(x,y)$, are computed as:

$$\Delta x(x,y) = x_c - x, \quad \Delta y(x,y) = y_c - y \tag{11}$$

where (x_c, y_c) is the ground truth center of the object. The loss for the center offset regression, L_{center}, is computed as:

$$\mathcal{L}_{center} = \frac{1}{N}\sum_{x=1}^{W/4}\sum_{y=1}^{H/4}\left(|\Delta\hat{x}(x,y)-\Delta x(x,y)| + |\Delta\hat{y}(x,y)-\Delta y(x,y)|\right)$$

(12)

where N is the number of object centers.

2. Width and Height Regression

In addition to the object center, the network predicts the dimensions of the bounding box, i.e., the width w and height h of the object. These are similarly estimated at each location (x, y). The true values $w(x, y)$ and $h(x, y)$ represent the ground truth width and height at location (x, y). The corresponding predictions are $\hat{w}(x, y)$ and $\hat{h}(x, y)$. The width and height regression loss, L_{size}, is computed as:

$$\mathcal{L}_{size} = \frac{1}{N} \sum_{x=1}^{W/4} \sum_{y=1}^{H/4} \left(|\hat{w}(x, y) - w(x, y)| + |\hat{h}(x, y) - h(x, y)| \right) \tag{13}$$

This ensures that the network accurately predicts the size of the object at each predicted location.

3. Yaw Angle Regression

The yaw angle θ, which represents the orientation of the object in the scene, is also predicted by the network. At each position (x, y), the yaw regression map outputs a predicted yaw angle $\hat{\theta}(x, y)$. The corresponding ground truth yaw angle is denoted as $\theta(x, y)$. The yaw angle regression loss, $\mathcal{L}_{\text{yaw}}$, is computed as the difference between the predicted and true yaw angles:

$$\mathcal{L}_{\text{yaw}} = \frac{1}{N} \sum_{x=1}^{W/4} \sum_{y=1}^{H/4} \left| \hat{\theta}(x, y) - \theta(x, y) \right| \tag{14}$$

where N is the number of valid object locations.

This loss function helps ensure that the network accurately regresses the object's orientation at each detected location. The use of this regression allows the network to handle rotated objects in the scene, improving the detection and localization performance, especially for non-axis-aligned objects.

4.6 Multi-Task Loss Function for Segmentation and Object Detection

The proposed MobilePIXOR framework incorporates a segmentation loss function to enhance the network's ability to differentiate between drivable and non-drivable areas. The segmentation loss consists of two components: the primary segmentation loss ($\mathcal{L}_{seg}$) and the auxiliary segmentation loss ($\mathcal{L}_{seg_aux}$), both computed using the cross-entropy loss function.

The segmentation loss is defined as follows:

$$\mathcal{L}_{seg} = \text{CE}(\hat{S}, S) = -\sum_{i} w_i S_i \log(\hat{S}_i) \tag{15}$$

where $\hat{S}$ represents the predicted segmentation map, S is the ground truth segmentation label, and w_i is the weight assigned to each class to handle class imbalance.

Similarly, the auxiliary segmentation loss is computed as:

$$\mathcal{L}_{seg_aux} = \text{CE}(\hat{S}_{aux}, S_{aux}) = -\sum_{i} w_i S_{aux,i} \log(\hat{S}_{aux,i}) \tag{16}$$

where $\hat{S}_{aux}$ and S_{aux} are the predicted and ground truth auxiliary segmentation maps, respectively.

The total loss function integrates both segmentation losses along with object detection losses, including heatmap loss ($\mathcal{L}_{heatmap}$), offset regression loss ($\mathcal{L}_{center}$), bounding box size loss ($\mathcal{L}_{size}$), and yaw angle loss ($\mathcal{L}_{yaw}$). The final multi-task loss function is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{heatmap} + \mathcal{L}_{center} + \mathcal{L}_{size} + \mathcal{L}_{yaw} + \lambda_1 \mathcal{L}_{seg} + \lambda_2 \mathcal{L}_{seg_aux} \tag{17}$$

where $\lambda_1 = 5$ and $\lambda_2 = 5$ are weighting coefficients that emphasize the importance of segmentation tasks relative to object detection.

5 Experiments

5.1 Dataset Preparation

Lidar3D_SEG&DET is a custom dataset designed for 3D LiDAR-based segmentation and object detection, focusing on drivable/non-drivable areas and detecting pedestrians and scooters in urban environments. It contains 1,491 training samples and 165 test samples, each with annotated 3D point clouds for instance-level classification and segmentation.

The dataset ensures comprehensive 3D coverage and includes voxelized representations to balance spatial accuracy and computational efficiency. For example, as shown in Figure 6, the dataset captures diverse urban scenarios, where pedestrians and scooters are accurately detected within drivable and non-drivable areas. With precise ground truth annotations, *Lidar3D_SEG&DET* serves as a benchmark for autonomous navigation, smart cities, and robotic perception.

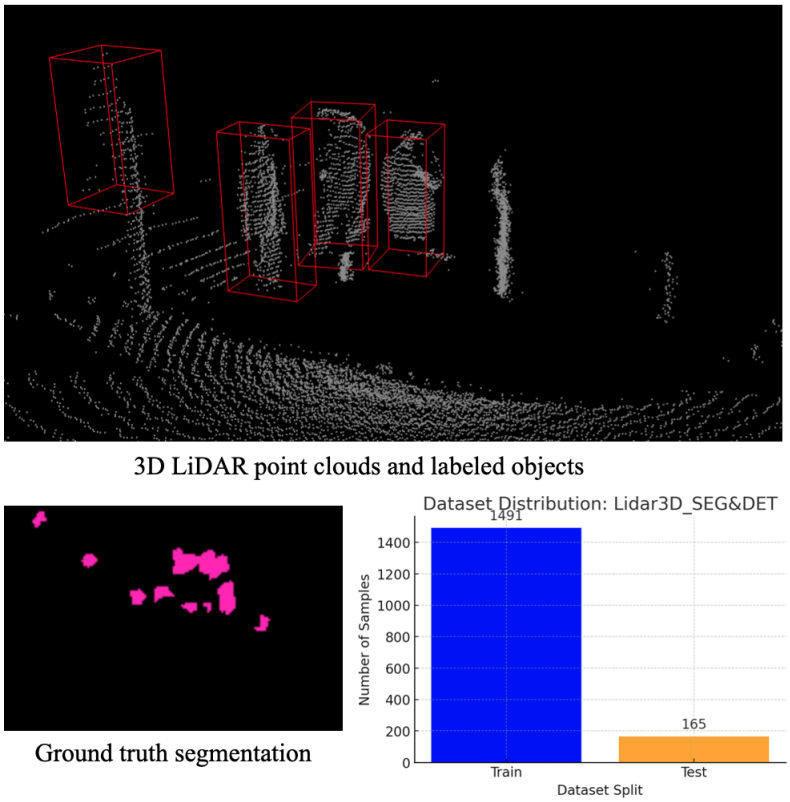


Fig. 6 Data distribution visualization for the custom dataset *Lidar3D_SEG&DET*.

5.2 Experimental Setup

The region of interest (ROI) for the point cloud is defined within the x-y plane as $[-6.0, 6.0] \times [-6.0, 6.0]$ meters, with a bird's-eye view (BEV) projection applied using a discretization resolution of 0.05m. The height span is set to $[-1.8, 1.2]$ meters, covering typical human heights while also accounting for variations in pedestrian postures and object elevation. The point cloud data is voxelized along the vertical axis into 30 bins, each with a bin width of 0.1m, facilitating effective feature extraction. Additionally, a reflectance channel is incorporated, resulting in a final input representation of dimensions $240 \times 240 \times 30$. Our approach differs from conventional detection methods, such as [17], which initialize network weights from pre-trained models. Instead, we train the model from scratch without relying on any pre-trained features.

The proposed attention-enhanced MobilePIXOR model is implemented using PyTorch [18]. The model supports semantic segmentation, and classification is conducted using a Gaussian encoding scheme. The training process is conducted on a macOS system, equipped with an M2-MPS (Metal Performance Shaders) and 32 GB of RAM. The optimizer utilized is AdamW [19], incorporating a one-cycle learning rate policy [20]. The initial learning rate is set to 0.002, with a momentum of 0.9 and a fixed weight decay of 0.001 to ensure convergence.

To enhance generalization, data augmentation is applied during training, incorporating random rotation within a $\pm 10°$ range ($p = 1.0$), scaling within the range $[0.95, 1.05]$ ($p = 1.0$), and translation with a scale factor of 0.4 ($p = 1.0$). These augmentation techniques help improve the model's robustness by introducing variations in the input data, thereby reducing overfitting and enhancing its ability to generalize across different scenarios.

Test Scenarios and Environments: The evaluation of the proposed model is conducted in diverse test scenarios, including urban and pedestrian-dense environments. These scenarios assess the model's robustness under different lighting conditions, object densities, and occlusions.

Evaluation Metrics: The performance of the proposed model is assessed using standard evaluation metrics, including accuracy, processing speed, and mean Intersection over Union (mIoU). The following key metrics are employed:

- **Intersection over Union (IoU):** IoU measures the ratio of the overlapping area between the predicted and ground-truth bounding boxes to their combined

Table 1 Comparative evaluation and accuracy assessment of 3D object detection proposed systems on the custom test set with varying IoU thresholds (0-6m).

Backbone Network	Avg Training Time/Epoch (s)	IOU = 0.5				IOU = 0.7			
		Precision	Recall	F1-score	AUC_{PR}	Precision	Recall	F1-score	AUC_{PR}
MobilePIXOR+GAU	338	0.9177	0.3593	0.5164	0.773	0.4677	0.1608	0.2393	0.1618
SE-MobilePIXOR+GAU	350	0.9323	0.3749	0.5348	0.7968	0.4687	0.1612	0.2399	0.1698
CBAM-MobilePIXORv1+GAU	380	0.9667	0.2991	0.4568	0.7326	0.5650	0.1486	0.2353	0.1884
CBAM-MobilePIXORv2+GAU	435	0.9628	0.2956	0.4523	0.7023	0.5449	0.1497	0.2349	0.1823
CBAM-MobilePIXORv3+GAU	366	0.9680	0.4381	0.6032	0.849	0.5909	0.2175	0.3180	0.2118

area. A higher IoU indicates better localization accuracy.

$$\text{IoU} = \frac{\text{Area of Overlap (AoO)}}{\text{Area of Union (AoU)}} \tag{18}$$

– **Precision:** Precision quantifies the proportion of correctly identified positive samples among all predicted positives. It evaluates how many of the detected objects are actual targets.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{19}$$

– **Recall:** Recall determines the proportion of correctly identified positive samples out of all actual positives. It reflects the model's ability to detect all instances of a given class.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{20}$$

– **Average Precision (AP):** The Average Precision (AP) is computed by integrating the **Precision-Recall (PR) curve, which represents the overall detection performance of the model. It is formally defined as:

$$AP = AUC_{PR} \int_0^1 P(R)\, dR \tag{21}$$

where $P(R)$ is the precision value at a given recall, and dR represents the change in recall values between consecutive points. In practice, this integral is approximated using the trapezoidal rule:

$$AP = \sum_{i=1}^{N-1} (R_i - R_{i-1}) \times \frac{P_i + P_{i-1}}{2} \tag{22}$$

where R_i, R_{i-1} are consecutive recall values, and P_i, P_{i-1} are the corresponding precision values. This numerical integration provides an effective approximation of the area under the Precision-Recall curve (PR-AUC).

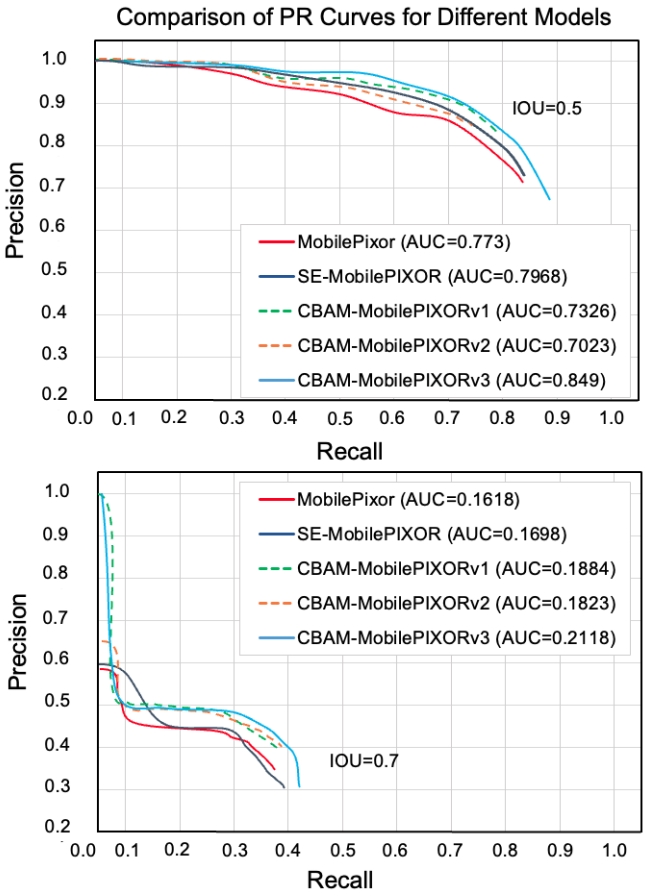


Fig. 7 Precision-Recall Curve Comparison of Different Models at IoU Thresholds 0.5 and 0.7 for the custom dataset *Lidar3D_SEG&DET*.

6 Results and Analysis

6.1 Object Detection Results

An evaluation was conducted on the Lidar3D SEG&DET dataset, and the results are presented in Table 1. The comparative analysis of multiple 3D object detection models across different IoU thresholds (0.5 and 0.7) indicates that CBAM-MobilePIXORv3+GAU outperforms other variants. It achieves the highest precision (0.9680 at IoU=0.5 and 0.5909 at IoU=0.7), F1-score (0.6032 at IoU=0.5 and 0.3180 at IoU=0.7), and AUC (0.849 at IoU=0.5 and 0.2118 at IoU=0.7). The integration

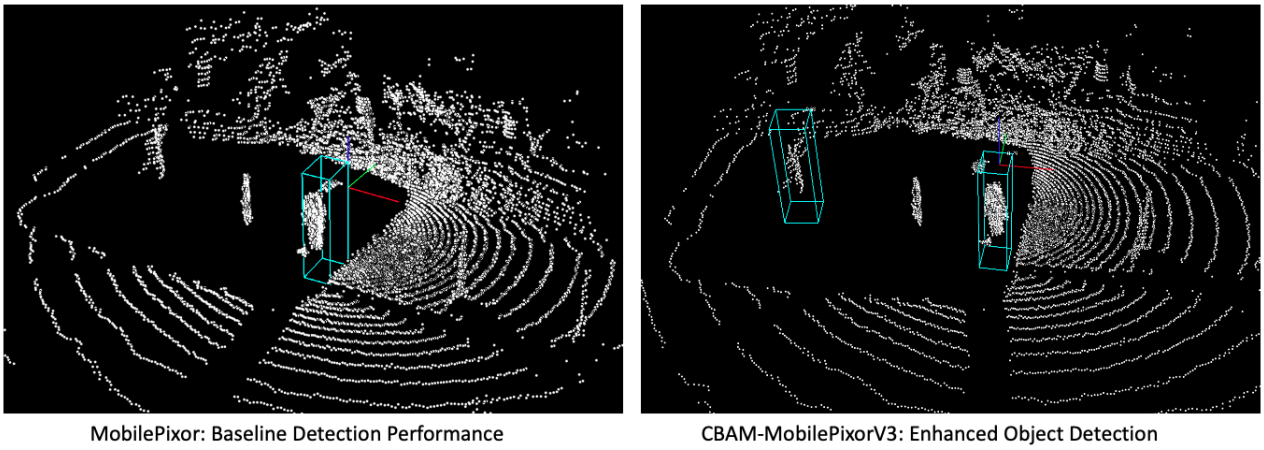


Fig. 8 Comparison of Object Detection Performance: MobilePIXOR vs. CBAM-MobilePIXORv3.

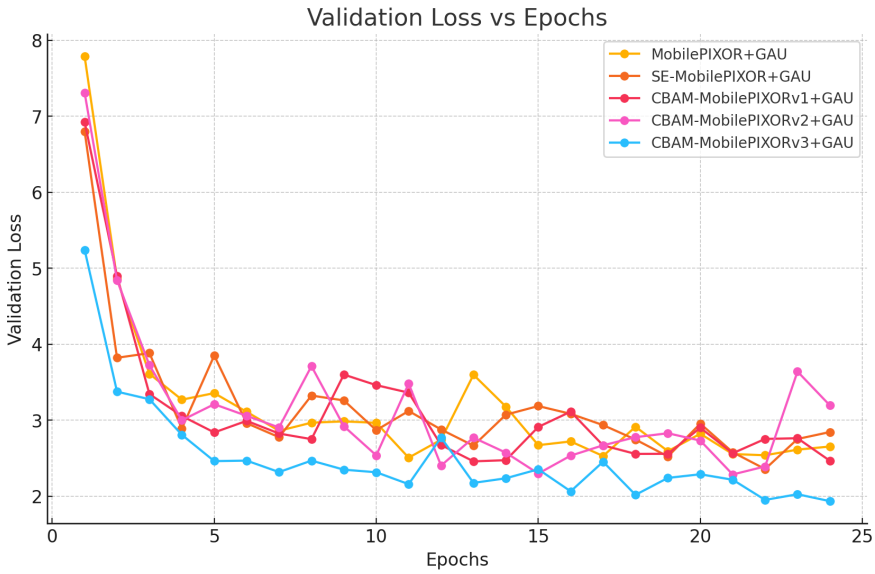


Fig. 9 Validation Loss Comparison of MobilePIXOR Variants over 24 Epochs.

of CBAM attention mechanisms enhances feature extraction, leading to more accurate and robust detection. While CBAM-MobilePIXORv2+GAU has the longest training time (435s per epoch), it does not significantly surpass CBAM-MobilePIXORv3+GAU, which demonstrates superior accuracy with a shorter training time (366s per epoch). SE-MobilePIXOR+GAU also shows competitive performance but remains behind CBAM-enhanced models in terms of F1-score and AUC. Notably, CBAM-MobilePIXORv3+GAU achieves a 8.9% higher AUC at IoU=0.5 and 23.6% higher AUC at IoU=0.7 compared to MobilePIXOR+GAU, which lacks attention mechanisms. The CBAM attention mechanism is placed at the final stage in the v3 architecture, allowing it to refine the entire feature map and effectively adjust the extracted information. Additionally, the presence of Skip Connections ensures efficient information flow throughout the network, enhancing feature preservation. This significant improvement highlights the impact of CBAM in enhancing object detection performance, particularly in challenging high-IoU scenarios. Fig. 7 illustrates the AP-AUC comparison of all methods, further emphasizing the benefits of incorporating attention mechanisms in 3D object detection models.

In Figure 8, object detection performance is compared on a representative sample from the dataset. The results indicate that the MobilePixor method detects only one object with limited coverage, whereas CBAM-MobilePIXORv3 shows significant improvements by identifying multiple objects with more precise bounding boxes. This demonstrates that the integration of CBAM enhances feature learning, leading to improved sensitivity and accuracy in LiDAR-based object detection.

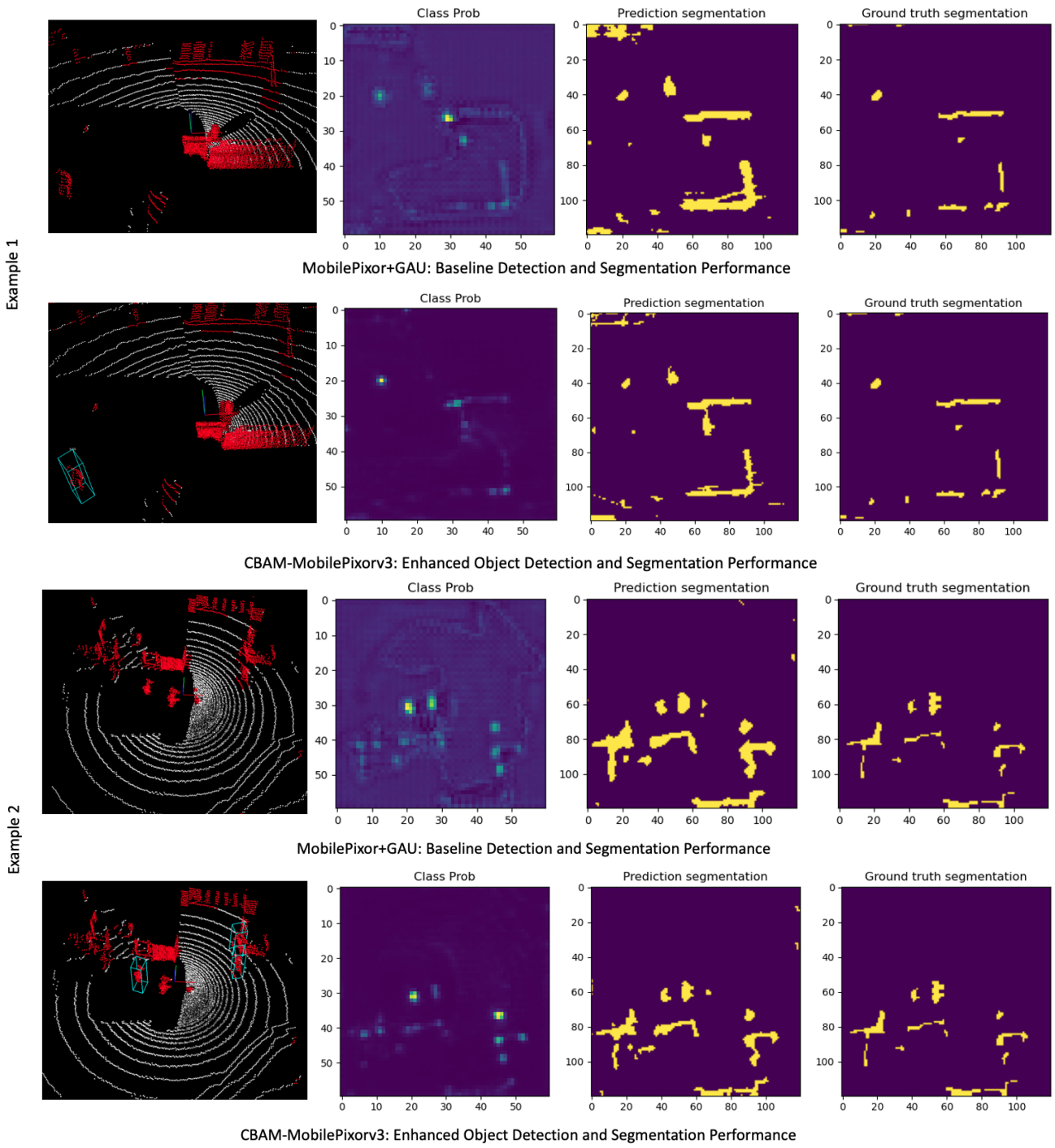


Fig. 10 Comparison of Detection and Segmentation Performance: MobilePIXOR+GAU vs. CBAM-MobilePIXORv3.

6.2 Free Space Segmentation Results

To evaluate the effectiveness of the proposed MobilePIXOR models in free-space segmentation, their validation loss is analyzed over multiple training epochs. Fig. 9 illustrates the validation loss comparison of the proposed MobilePIXOR models over 24 epochs. The total loss function (17) includes heatmap, center, size, yaw, segmentation, and auxiliary segmentation components. The CBAM-MobilePIXORv3+GAU model demonstrates the lowest validation loss, indicating superior generalization capability and enhanced performance in learning object boundaries and free-space segmentation. Fig. 10 visually compares the output of MobilePIXOR+GAU and CBAM-MobilePIXORv3 in a multi-task setting, including object detection and free-space segmenta-

Backbone Network	Mean IOU without aux_seg loss	Mean IOU segmentation with aux_seg loss
MobilePIXOR+GAU	0.441	0.467
SE-MobilePIXOR+GAU	0.482	0.514
CBAM-MobilePIXORv1+GAU	0.515	0.541
CBAM-MobilePIXORv2+GAU	0.527	0.553
CBAM-MobilePIXORv3+GAU	0.540	0.565

Table 2 Mean IOU segmentation results with and without auxiliary segmentation.

tion, demonstrating the superior performance of CBAM-MobilePIXORv3. The Prediction Segmentation and Ground Truth Segmentation columns reveal that CBAM-MobilePIXORv3 achieves a higher Intersection over Union (IoU), indicating more precise segmentation of object boundaries and free space. In the leftmost column, white points represent free space, while red points indicate obstacles, with CBAM-MobilePIXORv3 showing better object recognition and boundary segmentation. These improvements stem from the integration of CBAM (Convolutional Block Attention Module) for enhanced feature selection and GAU (Global Attention Unit) for improved generalization, leading to superior performance in both detection and segmentation tasks.

Table 2 presents the segmentation performance of various backbone networks with and without auxiliary segmentation loss. Among all models, CBAM-Mobile PIXORv3+GAU demonstrates the best segmentation capability for both free space and obstacles, achieving the highest Mean IOU of 0.565 when auxiliary segmentation loss is applied. Compared to the classical MobilePIXOR+GAU, which attains a Mean IOU of 0.441 without auxiliary loss, CBAM-MobilePIXORv3+GAU shows a relative improvement of 22.45%, highlighting the effectiveness of attention mechanisms in enhancing spatial feature extraction.

When auxiliary segmentation loss is introduced, CBAM-MobilePIXORv3+GAU further improves its Mean IOU from 0.540 to 0.565, achieving a 4.63% enhancement. Similarly, other CBAM-MobilePIXOR variants exhibit consistent improvements, with CBAM-MobilePIXORv2 +GAU increasing by 4.93% and CBAM-MobilePIXORv1 +GAU by 5.05%. The classical MobilePIXOR+GAU benefits the most from auxiliary segmentation loss, with a 5.90% gain, but still underperforms compared to attention-enhanced models. These results emphasize the advantage of combining attention mechanisms with auxiliary segmentation loss to refine obstacle and free space segmentation accuracy.

7 Conclusion

This study proposed a Multi-task Attention-Enhanced MobilePIXOR framework that effectively integrates CBAM (Convolutional Block Attention Module) and GAU (Global Attention Unit) into the MobilePIXOR architecture, enhancing both obstacle segmentation and pedestrian detection for real-time autonomous navigation. The incorporation of depthwise separable convolutions and Gaussian heatmaps allows for more efficient spatial feature extraction, improving the ability to differentiate between drivable and non-drivable areas while maintaining computational efficiency. Experimental results on the Lidar3D SEG&DET dataset validate the effectiveness of the proposed framework, demonstrating improved Intersection over Union (IoU) scores, more precise obstacle boundary segmentation, and enhanced object detection performance compared to existing models. The addition of an auxiliary segmentation loss further refines segmentation quality, contributing to better generalization across diverse environments. Future research will explore further optimizations, such as adapting the framework for larger-scale environments, enhancing its robustness in pedestrian-dense areas, and integrating multi-modal sensor fusion for improved perception. The findings from this study provide a strong foundation for real-time, perception-driven autonomous robotics, enabling safer and more efficient navigation in complex scenarios.

Acknowledgements
The authors would like to thank the support of time and facilities from FPT University, Can Tho, Vietnam, for this study.

Author Contributions
Hoang N. Tran, Cong T. Nguyen, and Truong T. Duong conceptualized the research idea and developed the algorithms. All authors contributed to the analysis and interpretation of the results. Minh L. Nguyen, Vinh D. Nguyen, and Cong T. Nguyen were responsible for drafting the initial manuscript. Hoang N. Tran supervised the research process and provided critical revisions to the manuscript. All authors reviewed and approved the final version of the paper.

Declarations

Ethics Approval Not applicable. (This study does not involve human participants, their data, or biological material). **Consent to Participate** Not applicable (This article does not involve human participants). **Con-**

sent for Publication Not applicable (This article does not involve human participants). **Conflicts of interest** The authors declare neither conflict of interest, nor competing interests.

References

1. Y. Zhou, O. Tuzel, V. Le, K. Kalogianni, and A. Pratkanis, "VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4490–4499, 2018.
2. H. Wang, Z. Chen, Y. Cai, L. Chen, Y. Li, M. A. Sotelo, and Z. Li, "VoxelRCNN-complex: An Effective 3-D Point Cloud Object Detector for Complex Traffic Conditions," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–12, 2022.
3. A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, "PointPillars: Fast Encoders for Object Detection from Point Clouds," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 12697–12705, 2019.
4. Y. Zou, H. Weinacker, and B. Koch, "Towards Urban Scene Semantic Segmentation with Deep Learning from LiDAR Point Clouds: A Case Study in Baden-Württemberg, Germany," *Remote Sensing*, vol. 13, no. 16, p. 3220, 2021, doi: 10.3390/rs13163220.
5. Chakraborty D, Dey EK, "Segmentation of LiDAR Point Cloud Data in Urban Areas Using Adaptive Neighborhood Selection Technique," *PLOS ONE* 19(7): e0307138. https://doi.org/10.1371/journal.pone.0307138 2021.
6. Dreissig, M., Piewak, F., & Boedecker, J. (2022). On the calibration of underrepresented classes in LiDAR-based semantic segmentation. *arXiv preprint arXiv:2210.06811.*
7. A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, "RangeNet++: Fast and Accurate LiDAR Semantic Segmentation," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 4213–4220, 2019.
8. T. Cortinhal, R. Toste, F. Cheein, and O. Aycard, "SalsaNext: Fast, Uncertainty-Aware Semantic Segmentation of LiDAR Point Clouds for Autonomous Driving," in *Proc. 23rd Int. Conf. Intelligent Transportation Systems (ITSC)*, pp. 646–653, 2020.
9. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
10. A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 7482–7491, 2018.
11. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.
12. B. Yang, W. Luo and R. Urtasun, "PIXOR: Real-time 3D object detection from point clouds", arXiv:1902.06326, 2019.
13. D. A. Nguyen, K. N. Hoang, N. T. Nguyen, D. A. Nguyen and H. N. Tran, "Enhancing Indoor Robot Pedestrian Detection Using Improved PIXOR Backbone and Gaussian Heatmap Regression in 3D LiDAR Point Clouds," in *IEEE Access*, vol. 12, pp. 9162-9176, 2024
14. J. Xu, S. Zuo, C. Wei, and W. Zhou, "LiSD: An Efficient Multi-task Learning Framework for LiDAR Segmentation and Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2024.
15. D. Chakraborty and E. K. Dey, "Segmentation of LiDAR Point Cloud Data in Urban Areas Using Adaptive Neighborhood Selection Technique," in *Proc. Int. Conf. Pattern Recognition (ICPR)*, 2024.
16. A. Khattar, S. Hegde, and R. Hebbalaguppe, "Cross-Domain Multi-task Learning for Object Detection and Saliency Estimation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 1–6, 2021.
17. X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, "Multi-view 3D object detection network for autonomous driving," *in Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2016, pp. 1907–1915.
18. A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in PyTorch," Tech. Rep., 2017.
19. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent., 2017, pp. 1–19.
20. S. Gugger. (2018). The 1Cycle Policy. [Online]. Available: https://sgugger.github.io/the-1cycle-policy.html